AI-BASED RIPENING STAGE DETECTION AND AUTOMATED SORTING SYSTEM FOR THE PADMA TOMATO VARIETY (*Solanum lycopersicum*) IN SRI LANKA

W.K. GAYAN

K.R.D.H. GUNAWARDHANA

P.L.H. HIRUSHAN

UVA WELLASSA UNIVERSITY OF SRI LANKA

[TO BE SUPPLIED: submission month] 2026

AI-BASED RIPENING STAGE DETECTION AND AUTOMATED SORTING SYSTEM FOR THE PADMA TOMATO VARIETY (*Solanum lycopersicum*) IN SRI LANKA

A dissertation submitted to the
Department of Computer Science and Informatics,
Faculty of Applied Sciences,
Uva Wellassa University
in partial fulfilment of the requirements for the award of the
Degree of Bachelor of Science (Honours) in Industrial Information Technology

by

WIJESINGHE KANKANAMALAGE GAYAN
KARUNARATHNA RALALAGE DON HASITHA GUNAWARDHANA
PATHIRANNEHELAGE LAKSHITHA HIRUSHAN

Department of Computer Science and Informatics

Faculty of Applied Sciences

Uva Wellassa University, Sri Lanka

[TO BE SUPPLIED: submission month] 2026

DECLARATION

We do hereby declare that the work reported in this dissertation was exclusively carried out by us under the supervision of Ms. D.P. Jayathunga, Dr. U.G.A.T. Premathilake and Ms. M.P.A.M. Rathnakumara. It describes the results of our own independent research except where due reference has been made in the text. No part of this dissertation has been submitted earlier or concurrently for the same or any other degree.

Date

W.K. Gayan
UWU/IIT/21/062
[TO BE SUPPLIED: email]

K.R.D.H. Gunawardhana
UWU/IIT/21/017
[TO BE SUPPLIED: email]

P.L.H. Hirushan
UWU/IIT/21/019
[TO BE SUPPLIED: email]

We endorse the declaration by the candidates.

Ms. D.P. Jayathunga
(Principal Supervisor)

Dr. U.G.A.T. Premathilake
(Co-supervisor)

Date

Date

Ms. M.P.A.M. Rathnakumara
(Co-supervisor)

Date

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our principal supervisor, Ms. D.P. Jayathunga, and to our co-supervisors, Dr. U.G.A.T. Premathilake and Ms. M.P.A.M. Rathnakumara, of the Department of Computer Science and Informatics, Uva Wellassa University, for their guidance, constructive criticism and continued encouragement throughout every stage of this research.

We thank the academic and technical staff of the Department of Computer Science and Informatics and of the Faculty of Applied Sciences for their instruction and for access to the facilities used in this work.

We are grateful to the management and staff of the tomato collecting centre in Bandarawela, Badulla District, for permitting the collection and photography of Padma tomato samples over the course of the study.

[TO BE SUPPLIED: add any further acknowledgements – for example workshop or fabrication assistance, and any funding source. Maximum one page in total.]

Finally, we thank our families and colleagues for their patience and support during the completion of this dissertation.

Contents

ABSTRACT

Post-harvest loss in the Sri Lankan fresh tomato supply chain is driven largely by manual grading, which is slow, subjective between operators and mechanically damaging to fruit. This research develops and evaluates an integrated computer vision and mechatronic sorting system for the locally cultivated Padma variety (*Solanum lycopersicum*). A dataset of 7,849 images was captured under controlled lightbox illumination at a commercial collecting centre in Bandarawela, annotated across five mutually exclusive classes (green, breaker, turning, red and defect) and split 70:20:10 into training, validation and held-out test subsets. A YOLOv8-medium detector was fine-tuned on a single 8 GB graphics processing unit, evaluated once against the held-out split, and compared with a YOLO26-medium model trained on identical data. A conveyor prototype was fabricated from a 25 mm box-section frame driven by a NEMA 23 stepper motor through a 3:1 timing-belt reduction, and fitted with an overhead camera, four infrared presence sensors and four servo-actuated diverter gates under ESP32 control. Rather than timing gate release from the full camera-to-gate distance, the firmware carries each classification through a cascaded stage queue and releases a gate only when the fruit breaks that stage's sensor beam, removing the dominant source of timing error. Ripening classes are mapped to a remaining shelf life obtained from a 31-day room-temperature observation study, and every classification is logged and displayed on a live operator dashboard.

[TO BE SUPPLIED: two to three sentences summarising the principal quantitative results and the main conclusion, to be written once Chapters 4 and 5 are complete]

Keywords: Padma tomato; YOLOv8; post-harvest sorting; mechatronic conveyor; shelf-life estimation

List of Figures

List of Tables

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ANN	Artificial Neural Network
CAD	Computer Aided Design
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CUDA	Compute Unified Device Architecture
DFL	Distribution Focal Loss
FIFO	First-In, First-Out
FN	False Negative
FP	False Positive
FPS	Frames Per Second
GPU	Graphics Processing Unit
HSV	Hue, Saturation, Value
IoU	Intersection over Union
IR	Infrared
KPI	Key Performance Indicator
LED	Light Emitting Diode
mAP	Mean Average Precision
MiB	Mebibyte
NIR	Near Infrared
R-CNN	Region-based Convolutional Neural Network
RGB	Red, Green, Blue
RPN	Region Proposal Network

RT-DETR	Real-Time Detection Transformer
SPP	Serial Port Profile
SVM	Support Vector Machine
TP	True Positive
USB	Universal Serial Bus
USDA	United States Department of Agriculture
VRAM	Video Random Access Memory
YOLO	You Only Look Once

Chapter 1

INTRODUCTION

1.1 Background of the Study

Post-harvest loss is a major source of inefficiency in the supply of perishable fresh produce, and it falls heaviest on growers in regions where cold-chain infrastructure is limited and grading is performed by hand. The tomato (*Solanum lycopersicum*) is particularly exposed. It is a climacteric fruit, so respiration and ethylene production continue after harvest and drive a rapid sequence of chlorophyll loss, carotenoid accumulation and cell-wall softening; once softening is advanced the fruit bruises easily and becomes vulnerable to fungal decay (Abekoon et al., 2024).

In Sri Lanka the locally recommended Padma variety is an important cash crop for smallholders in the Badulla, Nuwara Eliya, Kandy and Matale districts, valued for its resistance to bacterial wilt and its keeping quality (Abekoon et al., 2024). Fruit reaching collecting centres in these districts is still graded visually by hand. Operators sort bulk batches by the proportion of red surface colour into categories such as green, breaker, turning and red. Manual inspection of this kind is subjective, varies between operators and across a working shift, and requires repeated handling of every fruit. Two consequences follow. Misgraded overripe fruit entering a packed batch accelerates the ripening of its neighbours, and the repeated handling itself causes microscopic skin damage that provides an entry route for decay organisms (Abekoon et al., 2024).

Automated grading offers a route around both problems. Convolutional neural networks, and in particular single-stage detectors of the You Only Look Once (YOLO) family, now classify produce accurately and quickly enough to run on a moving conveyor (Moya et al., 2025; Terven and Cordova-Esparza, 2023), and when coupled to electronic actuators they can grade and divert fruit without a human touching it (Geetha et al., 2025). Applying such a system to a specific local cultivar is not, however, a matter of deploying an existing model. It requires a dataset built for that cultivar, a decision

policy suited to the way the fruit actually presents itself on a belt, and mechanical and control design that diverts fruit reliably without bruising it.

1.2 Problem Identification

The post-harvest handling of Padma fruit presents three linked problems.

At the biological level, the fruit softens quickly once ripening begins and is easily bruised by manual handling, so every additional handling step shortens the period during which the fruit remains saleable (Abekoon et al., 2024).

At the computational level, published ripeness models are trained predominantly on other cultivars, on other datasets and under other imaging conditions. Models built for one cultivar transfer poorly to another whose size, surface texture and colour progression differ; Nalupano et al. (2024) showed this directly when a general architecture had to be retrained specifically for the Philippine Kinalabasa variety. A further and less frequently reported difficulty is that a detector may learn the imaging conditions of a class rather than the class itself, so that a model performing well on an offline test split fails on live camera input. No public dataset exists for Padma fruit annotated for object detection across four ripening stages together with a defect class.

At the system level, most work reports a model evaluated offline on static images and stops there. Where a physical sorting rig is built, gate actuation is commonly timed from an assumed constant belt speed, which accumulates error as belt speed, load and drive slip vary. Furthermore, a grading decision by itself tells a packhouse manager only what a fruit is now, not how long it will remain saleable, and gives no batch-level view of the line.

1.2.1 Research Questions

1. How accurately can a single-stage object detector localise and classify Padma fruit into four ripening stages and a defect class from a purpose-built, cultivar-specific dataset, and which architecture within that family performs best on this task?

2. What decision policy converts a stream of per-frame detections into one reliable classification per fruit as it travels beneath a camera, given that the ripening classes lie on a continuous colour gradient?

3. What control architecture synchronises classification with physical gate actuation on a conveyor without relying on an assumed constant belt speed?

4. How can measured post-harvest stage durations for Padma fruit be converted into a remaining shelf-life value attached to each graded fruit, and reported alongside line-level operating indicators on a live dashboard?

1.2.2 Relevance and Significance of the Study

The economic argument rests on the loss that objective grading avoids. Replacing repeated manual inspection with a single automated pass reduces handling damage and removes the operator-to-operator variation that lets overripe fruit into packed batches (Abekoon et al., 2024).

The scientific contribution is threefold. First, the study produces an annotated Padma-specific detection dataset covering four ripening stages and a defect class, which does not currently exist. Second, it treats detection not as an offline metric but as one component of a working sensor-triggered sorting machine, and reports the integration honestly, including a documented case in which a detector learned the imaging conditions of a class rather than the class itself and had to be corrected. Reporting that failure and its correction is itself a contribution, because it is a failure mode that offline metrics conceal. Third, it links each grading decision to a remaining shelf-life value measured for this cultivar, and surfaces both on an operator dashboard, so that the output of the line is logistical information rather than only a physical sort.

1.3 Research Objectives

1.3.1 General Objective

To design, develop and evaluate an integrated, real-time detection and mechatronic sorting system for the Sri Lankan Padma tomato variety, which grades fruit into four ripening stages and a defect class, diverts it physically, associates each graded fruit with a measured remaining shelf life, and reports line-level operating indicators on a live dashboard.

1.3.2 Specific Objectives

1. To compile, annotate and preprocess a cultivar-specific image dataset representing four ripening stages (green, breaker, turning, red) and a defect class for the Padma variety.

2. To train and evaluate a YOLOv8-medium object detector on this dataset against a held-out test split, and to compare it under identical data conditions with a YOLO26-medium model in order to justify the deployed architecture on evidence rather than assertion.

3. To define and implement a per-fruit decision policy that aggregates per-frame detections across a tracked fruit into a single classification, including a deliberately conservative rule for the defect class.

4. To measure the duration of each ripening stage for Padma fruit under room-temperature storage, and to derive from those durations a per-class remaining shelf-life value applied at the moment of classification.

5. To design and construct a low-cost conveyor sorting rig controlled by an ESP32 microcontroller, in which gate actuation is released by an Infrared (IR) presence sensor at each gate rather than by an elapsed time computed from belt speed.

6. To develop a live dashboard that reports throughput, class distribution, defect ratio and mean remaining shelf life from the logged detection record.

1.4 Expected Outcomes

1. An annotated Padma-specific detection dataset covering four ripening stages and a defect class, available to support further localised research.
2. A trained detector for Padma ripening stage and defect condition, with performance reported on a held-out test split and against a comparison architecture.
3. A working conveyor sorting prototype using non-contact infrared triggering and servo-actuated diverter gates.
4. A per-class remaining shelf-life mapping derived from measured stage durations for this cultivar.
5. A live dashboard reporting throughput, class distribution, defect ratio and mean remaining shelf life over the logged detection record.

1.5 Structure of the Thesis

Chapter 1 sets out the background, identifies the problem, and states the research questions and objectives. Chapter 2 reviews ripening physiology and cultivar-specific grading, single-stage detection architectures, conveyor actuation and control, and shelf-life estimation, and identifies the gaps this study addresses. Chapter 3 describes the materials and methods: sample collection, dataset construction and annotation, model training and evaluation, the real-time decision policy, the hardware prototype, the control firmware, the shelf-life observation procedure, and the experimental and testing procedures. Chapter 4 reports and discusses the outcomes of those experiments. Chapter 5 draws conclusions and makes recommendations. Supporting records too extensive for the body text are placed in the appendices.

Chapter 2

LITERATURE REVIEW

2.1 Theoretical Background

An automated post-harvest grading and sorting machine sits across three fields: the ripening physiology that determines what is being graded, the detection architecture that performs the grading, and the control design that converts a grading decision into a physical action. This section reviews each in turn.

2.1.1 Ripening Physiology and Cultivar-Specific Grading

Ripening in tomato is climacteric. Under the influence of endogenous ethylene the exocarp loses chlorophyll while carotenoid pigments, principally lycopene, accumulate, so that surface colour moves from uniform green to deep red; in parallel, cell-wall pectins are progressively degraded and the fruit softens (Abekoon et al., 2024). Softening is the point at which the fruit becomes commercially fragile, because reduced firmness raises susceptibility to bruising and to the surface damage through which decay organisms enter.

Because surface colour tracks this internal progression, it is the basis of standard visual grading. The United States Department of Agriculture (USDA) standard defines six stages: green; breaker, at not more than 10 % red surface; turning, at 10–30 %; pink, at 30–60 %; light red, at 60–90 %; and red, above 90 % (United States Department of Agriculture, 1991). These stages are defined for grading and reporting, not for machine vision, and the two narrowest bands sit exactly where a camera is least reliable: under packhouse lighting, belt motion and specular reflection from the glossy cuticle, distinguishing 8 % red surface from 12 % is not a stable measurement.

Generic standards also travel poorly between cultivars. Nalupano et al. (2024) retrained a MobileNetV2 network specifically for the Philippine Kinalabasa variety, reaching

94 % accuracy but only 64 % specificity, which illustrates both that cultivar-specific training is necessary and that headline accuracy can conceal weak class separation. For Sri Lanka, Abekoon et al. (2024) studied the Padma variety directly. One hundred greenhouse-grown fruits, twenty-five from each of four districts, were held in a facility reproducing the mean conditions measured across sixty Sri Lankan marketplaces (29 °C, 80 % relative humidity, 810 lux) and photographed daily until deterioration, yielding 12,342 images. Their reported stage durations are reproduced in Table 2.1. That work establishes both a cultivar-specific colour-to-time relationship and the six-stage frame within which the present study defines its own classes.

Table 2.1: Post-harvest stage durations reported for the Padma variety under simulated market conditions

Maturity stage	**Days in stage**	**Post-harvest day range**
Green	2	0–2
Breakers	2	3–4
Turning	3	5–7
Pink	4	8–11
Light red	8	12–19
Red	12	20–31
Total	**31**	

Source: Abekoon et al. (2024), Table 4. Conditions: 29 °C, 80 % relative humidity, 810 lux; 100 fruits.

2.1.2 Real-Time Single-Stage Object Detection

Early agricultural vision systems used hand-crafted colour and texture descriptors with shallow classifiers. Santoso et al. (2024) compared RGB, HSV and CMYK representations using a correlation-coefficient matching method, finding CMYK the most discriminative with a correlation of 0.87. Liu et al. (2019) combined histogram-of-oriented-gradients descriptors with a Support Vector Machine (SVM) and a colour-based false-positive removal stage. Garcia et al. (2019) classified ripening stage from pixel colour statistics. These methods are computationally light, but they depend on descriptors chosen in advance and degrade when illumination, shadow or background departs from the conditions in which they were tuned (Ifmalinda et al., 2023).

Convolutional networks removed the need to choose descriptors by learning them from

data. Allo et al. (2025) trained a convolutional network on 3,600 public images across three ripeness classes, and Centino et al. (2025) obtained 95.52 % on a binary ripe/unripe task using background removal and colour-space conversion before classification. Waseem et al. (2025) pursued the same task under a computational constraint, using a ResNet-18 backbone with pruning and quantisation to keep maturity classification tractable on limited hardware. All three, however, classify a whole image. A sorting line needs localisation as well, because a frame may contain more than one fruit and each requires its own decision.

Object detectors supply that. Two-stage detectors such as Faster R-CNN first propose regions using a Region Proposal Network (RPN) and then classify them, which is accurate but too slow for a moving belt. Single-stage detectors of the YOLO family instead regress bounding boxes and class probabilities in one forward pass (Redmon et al., 2016). YOLOv8 is anchor-free and uses a cross-stage partial module with two convolutions for multi-scale feature fusion, giving the inference speed required for real-time operation (Terven and Cordova-Esparza, 2023).

Tomato-specific work concentrates on adapting this family to difficult imaging conditions. Yang et al. (2025) and Fan and Chai (2026) modified YOLOv8 for occlusion and lighting variation in greenhouse and orchard scenes; Huang et al. (2025) added a small-target detection head and multi-scale fusion to YOLOv10; Wang et al. (2026) and Ding et al. (2026) produced lightweight variants for deployment on constrained devices. Safira et al. (2026) compared YOLOv8 directly against a real-time detection transformer for tomato ripeness and provides the most direct published basis for preferring a YOLO detector for this task. Almost all of this work targets harvesting robots operating in the field or greenhouse, where the dominant problems are occlusion and cluttered backgrounds. A packhouse belt presents the opposite situation: a controlled background and a single fruit in view, but a decision that must be reached within the seconds the fruit is visible and then executed mechanically.

2.1.3 *Learned Imaging Conditions as a Confounding Cue*

A detector learns whatever feature separates its classes in the training data, including features that have nothing to do with the object. If the images of one class were captured under different lighting or against a different background from the rest, the network can learn that difference instead of the intended visual property, and will appear accurate on a test split drawn from the same distribution while failing on live input. The risk is highest when one class is scarce and is topped up from an external source, which is exactly the situation for a defect class. This is a well-recognised hazard, and the standard mitigations are to hold capture conditions constant across all classes and to increase minority-class representation by augmentation rather than by importing outside images (Shorten and Khoshgoftaar, 2019; Buslaev et al., 2020). It is nevertheless rarely reported in applied agricultural vision papers, which tend to publish final metrics rather than the diagnostic history behind them. Section 3.8.3 documents an instance encountered in the present study.

2.1.4 *Conveyor Actuation and Control*

Converting a classification into a physical diversion requires the gate to open at the moment the fruit reaches it. The straightforward approach computes the delay from belt kinematics. For gate i at distance d_i from the camera, with belt velocity v, inference latency t_{inf} and transmission latency t_{tx}, the delay before actuation is given in Equation (2.1).

$$t_{\text{act},i} = \frac{d_i}{v} - t_{\text{inf}} - t_{\text{tx}} \tag{2.1}$$

Equation (2.1) is elementary kinematics, and its weakness is the assumption that v is constant and known. On a real rig, belt tension, load, drive slip and motor torque all vary, and because d_i is the full camera-to-gate distance the resulting position error grows with distance down the line, affecting the last gate most. The alternative is to detect the fruit's arrival rather than predict it, by placing a presence sensor at each gate and holding pending classifications in a queue until the corresponding sensor is triggered. Existing

sorting rigs use belt sensors, though generally for a different purpose: in the system of Moya et al. (2025), sensors positioned along the conveyor detect fruit and trigger image capture, while the sorting servos act on the classification result. Geetha et al. (2025) likewise couple detection to a servo-driven sorting prototype. Using a sensor at each gate to release that gate, so that the elapsed-time term is eliminated from the actuation path entirely, is the specific design adopted here.

2.1.5 Embedded Integration and Wireless Command Transfer

Vision-based agricultural systems are commonly split between a host that performs inference and a microcontroller that drives actuators. Patria et al. (2025) used an ESP32-CAM with an artificial neural network on a mobile field robot, and Saxena et al. (2025) paired ESP32 sensor nodes with a Raspberry Pi running YOLOv8, using quantisation and pruning to gain a 35 % inference speed-up. Both illustrate the division of labour, though in both cases the microcontroller senses rather than actuates a sorting mechanism.

Where the link between host and controller is wireless, command delivery cannot be assumed. A serial-over-Bluetooth link on an operating rig is subject to intermittent loss, and in a sorting application a lost command is not merely a retransmission: it removes a fruit from the queue and misaligns every subsequent gate assignment. A command protocol for this purpose therefore requires per-command sequence numbering, explicit acknowledgement, and duplicate suppression so that a retransmitted command is acknowledged again but not acted on twice.

2.1.6 Shelf-Life Estimation and Operational Indicators

Shelf life is the period over which produce retains acceptable safety, firmness and sensory quality under defined storage conditions (Tarlak, 2023). Two approaches to estimating it appear in the literature. The first measures it directly, by observing a cohort of fruit under controlled conditions and recording how long each stage lasts, as Abekoon et al. (2024) did for the Padma variety. The second learns it, as Geetha et al. (2025) did

using deep models coupled to a sorting prototype. Non-destructive instrumental alternatives also exist: Borba et al. (2021) used portable Near Infrared (NIR) spectroscopy to assess tomato quality in the field, and Liu et al. (2024) fused multiple sensing modalities for maturity assessment. These give internal quality measures that colour imaging cannot, at the cost of instrumentation that a low-cost packhouse rig cannot carry.

Whichever route is taken, the estimate becomes operationally useful only when it is aggregated. A packhouse manager needs throughput, class distribution, defect ratio and mean remaining shelf life across a batch, not a value for one fruit (Geetha et al., 2025).

2.2 Gaps in Knowledge

Three gaps follow from the review above. They are stated narrowly, because adjacent work exists in each case.

No conveyor-integrated detection system exists for the Padma variety. Abekoon et al. (2024) characterised Padma ripening and post-harvest duration thoroughly, but the work is offline classification of static images of single fruits, with no detector, no localisation and no physical sorting. Conversely, conveyor-integrated systems such as Moya et al. (2025) are calibrated for other cultivars in other production contexts. No published work reports a detection model trained on Padma fruit and evaluated on an operating sorting line, and no public Padma detection dataset spanning four ripening stages and a defect class exists to build one from.

Gate release is generally predicted rather than detected. Belt sensors are already used in sorting rigs, but predominantly to trigger image capture (Moya et al., 2025). Actuation itself is typically timed from belt kinematics, per Equation (2.1), which makes sorting accuracy dependent on belt speed remaining constant and degrades progressively along the line. A control architecture in which each gate is released by its own presence sensor, with classifications held in a queue until arrival, removes that dependency but is not established in the tomato sorting literature.

Grade-to-shelf-life mapping and line-level reporting are rarely combined with an operating rig. This gap is the narrowest of the three, and must be stated carefully.

Geetha et al. (2025) do link deep-learning grading to shelf-life prediction and a servo-driven sorting prototype, so it cannot be claimed that no such system exists. What remains absent is a system in which the shelf-life values are measured for the specific cultivar being graded, rather than learned from generic data, and in which each classification and its associated remaining shelf life are logged to a persistent record that drives live batch-level operating indicators. It is that combination, for this cultivar, that the present study addresses.

Chapter 3

MATERIALS AND METHODS

3.1 Introduction

This chapter describes the materials, techniques and procedures used to develop an artificial intelligence based ripening stage detection and sorting system for the Padma tomato variety. The work combines an applied computer vision procedure covering sample collection, dataset construction, model training and evaluation, with an engineering prototyping procedure covering mechanical, electronic and firmware development. The two are brought together in a conveyor prototype that detects a fruit on a moving belt, classifies its ripening stage or defect condition, associates an estimated remaining shelf life with that classification, and diverts the fruit into the corresponding output stream.

The research was carried out jointly and is divided into three interdependent sub-systems: detection model development with real-time inference and hardware integration; shelf-life estimation and performance-indicator monitoring; and the mechanical conveyor prototype with its actuation. All three are documented here. The chapter follows the order in which the work was carried out. Outcomes are reported in Chapter 4, and established standard procedures, full component data and raw records are placed in the appendices.

3.2 Overall Research Methodology

The research followed an applied, iterative development methodology rather than a single-pass linear pipeline, so that the dataset, the annotation convention and the model could each be revised in response to evidence obtained during evaluation. The stages, in the order carried out, were: problem definition; sample collection; image acquisition; dataset preparation, annotation and splitting; preprocessing and augmentation; model

development, training, validation and testing; performance evaluation, with dataset and annotation correction and retraining where evaluation exposed a systematic weakness; hardware prototype design and fabrication; software development; hardware and software integration; real-time detection and classification; sorting decision and gate actuation; and system-level evaluation. The workflow, including the correction feedback loop, is shown in Figure 3.1.

[TO BE SUPPLIED: Figure 3.1 – overall research methodology flowchart, including the correction feedback loop from performance evaluation back to dataset and annotation correction]

Figure 3.1: Overall research methodology, from problem definition to system level evaluation

3.3 Research Materials

3.3.1 Tomato Samples

Padma variety tomatoes were the biological material of the study. Fruits were obtained from a commercial tomato collecting centre in Bandarawela, Badulla District, to which Padma tomatoes are delivered from several cultivation areas across the region. Sourcing from a multi-farm collecting centre rather than a single farm captures natural variation in fruit size, surface texture, maturity and handling damage within each class. Fruits were visually sorted at the point of capture into the five categories defined in Section 3.7. The materials used are summarised in Table 3.1.

Table 3.1: Materials and samples used in the research

Material	Description	Purpose
Padma tomato fruit	Fresh fruit at four ripening stages, and fruit with visible cracking, bruising or rot. Obtained from a commercial collecting centre, Bandarawela, Badulla District	Image dataset (Section 3.6), shelf-life observation (Section 3.18) and conveyor testing (Section 3.20)
Publicly sourced defect images	Third-party defect images used in an earlier dataset version, captured under uncontrolled background and illumination	Removed during quality control; not part of the final dataset (Section 3.8.3)

[TO BE SUPPLIED: total number of fruits photographed, the capture session dates, and the inclusion and exclusion criteria applied to each class. Sample-level records go in Appendix A]

3.3.2 Hardware Components

The prototype is a belt conveyor with a fixed overhead camera and four servo-actuated sorting gates. Its principal components are listed in Table 3.2; minor electronic components, the full pin assignment and wiring diagrams are given in Appendix C.

Table 3.2: Principal hardware components of the conveyor sorting prototype

Component	Model / specification	Qty
Microcontroller	ESP32 development board (DOIT DEVKIT V1); runs the sorting firmware and drives the motor driver, servos and sensors	1
Drive motor	NEMA 23 stepper motor	1
Motor driver	DM542 step-and-direction driver, commanded at 0–2,000 steps per second	1
Driver power supply	24 V, 5 A	1
Logic power supply	12 V, 2 A adapter, stepped down for the logic rail	1
Gate actuators	MG995 servo with printed diverter blade, one per ripening class	4
Presence sensors	Infrared reflective sensor, one at each gate position	4
Power transmission	20-tooth motor pulley, 60-tooth roller pulley and closed-loop timing belt, giving a 3:1 reduction	1 set
Frame and mounts	25 mm (1 inch) box-section frame, six bracket-and-bearing assemblies, printed servo and sensor brackets, adjustable single-axis camera mount	1 set
Speed control	Potentiometer, read by the microcontroller to set belt speed	1
Camera	USB high-definition webcam, operated at 960×720 pixels and 30 FPS	1
Host computer	Workstation with NVIDIA GeForce RTX 3070 Ti, 8,192 MiB VRAM	1

[TO BE SUPPLIED: confirm the exact webcam model, and state how the four MG995 servos are powered – whether from the stepped-down logic rail or a separate regulator. Four MG995 units can draw several amperes together at stall, so the arrangement should be stated explicitly]

3.3.3 Software and Technologies

The software environment is listed in Table 3.3. Exact versions are recorded because the behaviour of the training pipeline depends on the specific releases used.

Table 3.3: Software, frameworks and libraries used in the research

Software	Version	Purpose
Python	3.11.9	Training, inference, dashboard and hardware bridge
PyTorch and torchvision	2.11.0 and 0.26.0 (CUDA 12.8)	Deep learning framework
Ultralytics	8.4.104	Detection architecture, training, validation, inference
OpenCV	5.0.0.93	Camera capture, frame processing, box rendering
Albumentations	Stable release at time of use	Offline bounding-box-aware augmentation
Streamlit and Plotly	1.60.0 and 6.9.0	Operator dashboard and charts
SQLite	Bundled with Python 3.11.9	Local detection and indicator database
NumPy, pandas, PyYAML, pyserial	2.4.4, 3.0.5, 6.0.3, 3.5	Numerical work, tabular data, configuration, serial link
Roboflow	Web platform	Annotation and versioned dataset export
PlatformIO, Arduino framework, ESP32Servo	Project pinned; ESP32Servo 3.0.5	Firmware build, upload and servo control
SolidWorks and Onshape	Computer aided design platforms	Three-dimensional design of the prototype

3.4 Sampling Method and Sample Collection

Samples were collected using a purposive sampling strategy. Rather than drawing fruits at random, images were deliberately captured across all five target categories so that the dataset contained adequate representation of every category the model was required to learn, including the comparatively uncommon defect condition. Purposive sampling is appropriate here because the objective is model generalisation across defined visual categories for a supervised detection task, not population-level inference about the distribution of ripening stages in the crop (Etikan et al., 2016).

Collection began in January 2026 and continued over several sessions. All fruits were photographed in a white-box lightbox under LED illumination, which standardises background and illumination between sessions and removes lighting as an uncontrolled source of variation between classes. This control is essential rather than cosmetic: as

Section 3.8.3 records, a class captured under different conditions from the rest allowed the detector to learn the imaging conditions instead of the fruit.

3.5 Image Acquisition

Dataset images were acquired with the fruit placed against a white background inside the lightbox. The same arrangement was used for all five classes in the final dataset.

The deployment path is separate. For real-time inference the system uses a USB webcam connected to the host computer, opened at 960×720 pixels and 30 FPS. After initialisation, automatic exposure and white balance are allowed approximately 1.5 seconds to settle before any frame is passed to the model, because frames captured during this interval carry colour casts severe enough to alter the predicted class.

[TO BE SUPPLIED: camera model, native resolution and file format used for dataset capture; the camera-to-fruit distance and viewing angle; whether several orientations of the same fruit were deliberately captured and how many; and any image-quality rejection step applied at capture time]

[TO BE SUPPLIED: Figure 3.2 – lightbox capture setup and the conveyor-mounted camera arrangement]

Figure 3.2: Image acquisition setup for dataset capture and the conveyor mounted camera arrangement

3.6 Dataset Description

The dataset is a versioned export produced through the Roboflow annotation and dataset management platform (Dwyer et al., 2024), exported in the YOLOv8 label format. Its composition is summarised in Table 3.4 and the per-class distribution of annotated object instances in Table 3.5. Instance counts differ from image counts because an image may contain more than one annotated fruit. The offline augmentation supplement is listed in a separate column throughout, so that synthetic data is never confused with captured data.

Table 3.4: Composition of the dataset used for model development

Property	**Value**
Number of classes	5 (breaker, defect, green, red, turning)
Training images	5,498 captured, plus 668 offline-augmented defect images (Section 3.9)
Validation images	1,574
Test images (held out)	777
Total distinct captured images	7,849
Annotated instances, captured images only	7,920
Annotated instances, including the augmentation supplement	8,816
Annotation format	YOLO normalised bounding box: class index, x centre, y centre, width, height
Model input resolution	640×640 pixels after resizing
Hosting and version control	Roboflow versioned export, Creative Commons Attribution 4.0

Table 3.5: Per class annotated object instance counts by dataset split

Class	**Train (captured)**	**Train (augmented)**	**Validation**	**Test**	**Total**
breaker	1,137	0	326	191	1,654
defect	448	896	137	66	1,547
green	1,380	0	429	191	2,000
red	1,434	0	395	190	2,019
turning	1,138	0	309	149	1,596
Total	**5,537**	**896**	**1,596**	**787**	**8,816**

The defect class holds only 66 annotated instances in the test split, against 149–191 for each ripening class. Any performance figure reported for the defect class therefore carries appreciably wider uncertainty than the others, and this is accounted for in the evaluation procedure of Section 3.13.

[TO BE SUPPLIED: Figure 3.3 – representative captured images for each of the five target classes]

Figure 3.3: Representative sample images for each of the five target classes

3.7 Ripening Stage Classification Criteria

Five mutually exclusive target classes were defined: four ripening stages and one quality class. Defect was defined as a separate class rather than as an attribute of a ripening class, because damaged fruit requires the same handling irrespective of its colour.

The four ripening stages are obtained by merging adjacent pairs of the six USDA stages (United States Department of Agriculture, 1991), as reproduced in Table 2.1. Two considerations drove the merge. First, four classes map directly onto a four-gate sorting mechanism. Second, and more importantly, the two narrowest USDA bands sit exactly where camera-based estimation of red surface proportion is least stable, so merging them produces boundaries that a detector can separate reliably under belt motion and specular reflection. The correspondence is set out in Table 3.6, and the resulting criteria in Table 3.7.

Table 3.6: Correspondence between the six standard maturity stages and the four operational classes used in this research

Standard stage	**Standard red surface**	**Operational class**	**Operational red surface**
Green	0 %	Green	0 %
Breaker	$\leq$ 10 %	Breaker	$<$ 30 %
Turning	10–30 %		
Pink	30–60 %	Turning	30–80 %
Light red	60–90 %		
Red	$>$ 90 %	Red	$>$ 80 %

Standard stages and their boundaries from United States Department of Agriculture (1991). The operational upper boundary of the turning class is set at 80 % rather than 90 % so that fully coloured fruit is captured by the red class under belt lighting.

Table 3.7: Ripening stage and quality classes used in this research

Class	Stage	Identification characteristics
Green	Unripe (Stage 1)	Entirely green skin, with no red or orange colouration
Breaker	Onset of ripening (Stage 2)	Green to yellow transition, with less than 30 % of the surface red or pink
Turning	Mid ripening (Stage 3)	Between 30 % and 80 % of the surface red, with mixed green, orange and red patches
Red	Fully ripe (Stage 4)	More than 80 % of the surface red, with uniform ripe colouration
Defect	Rejected quality	Visible cracking, bruising or rot; may occur at any stage and receives no ripening label

3.8 Image Annotation

3.8.1 Annotation Tool and Format

All images were annotated in Roboflow (Dwyer et al., 2024) and exported in the YOLOv8 format, in which each image is accompanied by a plain-text label file containing one line per annotated object. Each line specifies a class index followed by four normalised bounding-box coordinates, namely x centre, y centre, width and height, expressed as fractions of the image dimensions.

3.8.2 Annotation Convention

Every fruit visible in an image was annotated individually with its own whole-fruit bounding box and a single class label, so that an image containing several fruits carries a corresponding number of annotations. For the defect class the convention is a whole-fruit bounding box rather than a polygon isolating the blemished region, labelled defect irrespective of the underlying ripening colour. A uniform box convention across all classes is essential, because a detector implicitly learns the object-scale statistics of each class: a class annotated with systematically smaller boxes than the others is being taught a different object size as well as a different appearance.

[TO BE SUPPLIED: Figure 3.4 – annotated image showing whole-fruit bounding boxes on mixed classes]

Figure 3.4: Example of whole fruit bounding box annotation for a multiple fruit image with mixed classes

3.8.3 Annotation Quality Control and Correction

Annotation was subject to a review and correction cycle. Two corrections applied during that cycle determined the final composition of the dataset, and both are documented because they materially shaped the method.

Inconsistent box geometry. Defect images originally drawn from a public dataset had been annotated with polygons enclosing only the blemished region, while the project's own images used whole-fruit boxes. Every such image was re-annotated under the whole-fruit convention so that box geometry was consistent across all five classes.

Imaging conditions acting as a class cue. Even after box geometry was made consistent, live camera testing showed healthy breaker, turning and red fruit being classified as defect, a failure not visible in the offline metrics. The cause was that the defect images were still drawn largely from a public dataset with uncontrolled backgrounds and illumination, while the four ripening classes had been captured entirely inside the lightbox. The detector had partly learned the imaging conditions as the distinguishing feature of the defect class, exactly the confound described in Section 2.1.3. The correction was to remove the mismatched images entirely, capture and annotate new defect images under the same lightbox conditions and convention as the other classes, and apply the class-balancing augmentation of Section 3.9 to compensate for the smaller number of real defect images remaining. The dataset described in Section 3.6 is the corrected export.

Annotation reliability. The class boundaries in Table 3.7 are defined by the proportion of red surface, estimated visually rather than measured instrumentally. To establish the repeatability of that judgement, a random sample of images was re-annotated independently and agreement between the two passes computed.

[TO BE SUPPLIED: the size of the re-annotated sample, and the resulting agreement statistic. If no reliability check has yet been performed, it should be carried

out before submission: with visually judged percentage boundaries and no colorimetric measurement, label reliability is otherwise unevidenced]

The detailed annotation procedure is given in Appendix A.

3.9 Image Preprocessing and Augmentation

Resizing. All images are resized to 640×640 pixels at both training and inference time, since a fixed input size is required for batched processing and this is the standard operating resolution of the architecture. Pixel-value scaling is handled internally by the framework, and no background subtraction or segmentation step is applied.

Training-time augmentation. Augmentation was applied on the fly by the framework's built-in pipeline, using the parameter values listed in Table 3.9. It combines hue, saturation and value jitter, rotation, translation and scaling, horizontal flipping with vertical flipping disabled because fruit on a belt is not observed inverted, mosaic augmentation closed for the final ten epochs, mixup, random erasing and automated policy selection. Augmentation of this kind is standard practice for improving generalisation from limited data (Shorten and Khoshgoftaar, 2019), and is applied here because live conveyor conditions differ from the lightbox conditions of capture.

Offline augmentation of the defect class. After the corrections of Section 3.8.3, the defect class retained 448 training instances against 1,137–1,434 for each ripening class. A separate offline step was therefore applied to its training images using the Albumentations library (Buslaev et al., 2020), which transforms bounding boxes together with the image. Flip, rotation, scale, brightness and hue-jitter transforms were applied to the 334 defect-only training images, producing two augmented copies of each and yielding 668 additional images carrying 896 additional instances. These were referenced only from the training split. The validation and test splits were left untouched, so that every reported evaluation figure is measured against captured images alone.

Dataset audit. An automated audit runs before every training run and halts it on a structural fault. It verifies the class-name order, confirms that a test split is present, computes per-split and per-class instance counts, flags orphaned images and empty label

files, detects duplicate images leaking across splits by message-digest hashing, and flags any class whose mean relative bounding-box area falls below 35 % of the median across classes. The last check was added specifically after the box-geometry fault described above, so that an inconsistent annotation convention cannot pass unnoticed into training again.

Limitation of the duplicate check. Message-digest hashing detects only byte-identical images. Because several photographs were taken of each fruit, and the export was split at the level of the individual image, near-duplicate views of the same physical fruit could in principle be distributed across splits, which would optimistically bias the reported metrics.

[TO BE SUPPLIED: state whether images of one physical fruit were confined to a single split. If they were not, either re-split the dataset grouped by fruit identity and retrain, or report this explicitly as a limitation on the reported metrics in Chapter 5]

[TO BE SUPPLIED: Figure 3.5 – preprocessing and augmentation pipeline, showing the offline defect branch separately]

Figure 3.5: Image preprocessing and augmentation pipeline, including the offline defect class branch

3.10 Dataset Splitting

The dataset was divided into training, validation and test subsets at export time in the proportions given in Table 3.8, corresponding to a conventional 70:20:10 division. The training set fits model parameters, the validation set monitors generalisation and selects the best checkpoint through early stopping, and the test set is used once to obtain an unbiased estimate of performance. It was kept fully held out, including from the offline augmentation.

Table 3.8: Dataset split used for model development

Subset	Images	Percentage	Role
Training	5,498	70.0 %	Fitting of model parameters; supplemented with 668 augmented defect images
Validation	1,574	20.1 %	Monitoring generalisation, early stopping and checkpoint selection
Test	777	9.9 %	Held out from training and checkpoint selection; used once for final evaluation
Total	**7,849**	**100 %**	

3.11 Model Development

The system uses YOLOv8, a single-stage convolutional object detection architecture, for simultaneous localisation and classification, in the medium-capacity YOLOv8m variant, initialised from publicly released pre-trained weights and fine-tuned on the project dataset. The YOLO family performs localisation and classification in a single forward pass, predicting bounding boxes and class probabilities directly from the full image without a separate region-proposal stage (Redmon et al., 2016; Terven and Cordova-Esparza, 2023).

This architecture was selected because a frame above the conveyor may contain more than one fruit at different ripening stages, and the sorting logic requires a bounding box, class label and confidence score for each at a frame rate compatible with belt speed. Published comparison supports the choice for this task (Safira et al., 2026). The medium variant was chosen as a compromise between accuracy and the memory available on the 8 GB host. No post-hoc classification stage is applied: the decision is the class prediction of the detector itself, aggregated across frames by the policy of Section 3.14.

Because architecture choice ought to rest on evidence from this dataset rather than on published results from others, a second detector was trained under identical data conditions for comparison, as described in Section 3.20, Experiment 2.

[TO BE SUPPLIED: Figure 3.6 – single stage detection workflow of the YOLOv8 architecture: input frame, backbone and neck feature extraction, detection head, boxes with class and confidence]

Figure 3.6: Simplified single stage detection workflow of the YOLOv8 architecture

3.12 Model Training

Training was performed locally rather than on a cloud service, on a workstation with an NVIDIA GeForce RTX 3070 Ti GPU of 8,192 MiB VRAM, running Python 3.11.9, PyTorch 2.11.0 built against CUDA 12.8 and Ultralytics 8.4.104. Because a single 8 GB device was used, the training script performs automatic batch-size selection and a pre-flight check of available video memory before the run begins. The configuration applied is given in Table 3.9, and the complete configuration file is reproduced in Appendix B.

Table 3.9: Training configuration used for model development

Parameter	**Value**
Base architecture	YOLOv8m, initialised from pre-trained weights
Training data	5,498 captured images and 668 offline-augmented defect images
Maximum epochs and early stopping	150 epochs; halts after 25 epochs without validation improvement
Batch size and input size	Automatic selection; 640×640 pixels
Optimiser	Automatic selection
Initial learning rate and final factor	0.01 and 0.01
Momentum and weight decay	0.937 and 0.0005
Warm-up epochs	3.0
Loss term weights	Box 7.5, classification 0.5, Distribution Focal Loss (DFL) 1.5
Colour augmentation	Hue 0.015, saturation 0.7, value 0.4
Geometric augmentation	Rotation 15°; translation 0.1; scale 0.5; horizontal flip 0.5; vertical flip disabled
Composite augmentation	Mosaic 1.0, closed for the final 10 epochs; mixup 0.15; random erasing 0.4; automated policy selection

The training procedure runs in five steps: a pre-flight check of the framework installa-

tion, device availability and free video memory; the dataset audit of Section 3.9, which halts the run if a structural fault is found; training with the configuration of Table 3.9, evaluated against the validation split after every epoch, with the best checkpoint written to disk and intermediate checkpoints saved every five epochs; early stopping once the patience limit is reached, restoring the best checkpoint as the final model; and a separate evaluation pass of that checkpoint against the held-out test split.

3.13 Validation and Evaluation Procedure

The validation split was used throughout training to monitor generalisation after every epoch, independently of the training data. Early stopping on a held-out split, rather than manual inspection of a learning curve, is the mechanism used to guard against overfitting, because it applies a fixed and reproducible criterion. The test split was evaluated exactly once, so that the figures reported in Chapter 4 are an unbiased estimate rather than values the model was tuned towards.

The standard object detection metrics defined below were computed per class and as a class-averaged mean (Padilla et al., 2020). Precision, in Equation (3.1), is the proportion of predicted detections of a class that are correct.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{3.1}$$

where TP is the number of true positive detections and FP the number of false positive detections. Recall, in Equation (3.2), is the proportion of actual instances of a class that were successfully detected.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{3.2}$$

where FN is the number of false negatives, that is, ground-truth instances the model failed to detect. The F_1 score, in Equation (3.3), is the harmonic mean of precision and recall, and was used to identify the confidence threshold at which the two are jointly optimised.

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.3}$$

Mean average precision, in Equation (3.4), is the mean across classes of the average precision of each class, where the average precision AP_i of class i is the area under its precision–recall curve.

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} AP_i \tag{3.4}$$

where N is the number of classes. Two variants were computed: mAP@50, which counts a detection as a true positive when its box overlaps the ground truth by at least 0.50 Intersection over Union (IoU) and the class matches; and mAP@50–95, which averages the metric over thresholds from 0.50 to 0.95 in steps of 0.05 and therefore also rewards precise localisation. A confusion matrix was generated as a diagnostic to identify which classes are confused with one another; as Section 3.8.3 shows, that diagnostic revealed a fault the aggregate metrics did not.

Uncertainty. Because the test split is finite, and the defect class in particular contributes only 66 instances, point estimates alone would overstate the precision of the comparison in Experiment 2. A non-parametric bootstrap over the test images was therefore used to obtain a 95 % confidence interval for each reported metric, resampling the test set with replacement 1,000 times and recomputing the metric on each resample. Differences between model configurations are treated as meaningful only where the intervals do not overlap.

System-level indicator data, comprising fruits processed, class distribution, defect ratio, throughput and mean estimated shelf life, is computed by direct aggregation over the detection log and is descriptive rather than inferential.

3.14 Real Time Detection Process

The procedure applied at inference time proceeds through the following steps.

1. **Frame capture.** A live frame is captured from the webcam at 960×720 pixels and 30 FPS.

2. **Preprocessing.** The frame is resized to the 640×640 pixel model input size.

3. **Inference.** The frame is passed through the trained model at a confidence threshold of 0.45, selected as the approximately F_1-optimal operating point identified from Equation (3.3) and the corresponding curve.

4. **Non-tomato object filtering.** A secondary general-purpose detector, pre-trained on a large-scale everyday-object dataset (Lin et al., 2014), is run on the same frame at a confidence threshold of 0.35 and an IoU threshold of 0.3, to identify distractors such as bottles, cups, telephones and hands. Any tomato detection overlapping a distractor is suppressed. This stage is necessary because the tomato model has no negative class for the objects that appear near an operating conveyor, and would otherwise assign one of its five labels to whatever enters the frame.

5. **Tracking and vote accumulation.** A tracking state machine with idle and tracking states follows a single fruit across consecutive frames, assuming single-file flow, and accumulates one class vote per frame. A track closes after 0.6 seconds without detection, and a minimum of two frames is required for a track to count.

6. **Classification finalisation.** When a track closes, the final class is decided by a confidence-weighted vote, given in Equation (3.5), over the set F of frames in the track.

$$\hat{c} = \arg\max_{c \in C} \sum_{f \in F} s_{f,c} \cdot \mathbf{1}\left[c_f = c\right] \tag{3.5}$$

where C is the set of five classes, c_f is the class predicted in frame f, and $s_{f,c}$ is the confidence assigned to class c in that frame. Aggregating over a track rather than trusting a single frame is what makes the decision robust to the momentary misclassifications that occur as a fruit rotates or passes through a specular highlight.

An override applies to the defect class. Defect is selected as the final class only if the three conditions in Equation (3.6) hold simultaneously:

$$n_d \geq 2 \quad \wedge \quad \frac{n_d}{|F|} \geq 0.35 \quad \wedge \quad \min_{f \in F_d} s_{f,\text{defect}} \geq 0.80 \tag{3.6}$$

where $F_d \subseteq F$ is the subset of frames voting defect and $n_d = |F_d|$. The rule is deliberately conservative, and it encodes a cost asymmetry rather than a statistical one: wrongly rejecting a saleable fruit removes revenue permanently, whereas passing a marginal fruit is recoverable downstream. It also provides a second line of defence against any residual trace of the confound identified in Section 3.8.3.

Finally, the finalised class, its confidence and the shelf-life value from the lookup of Section 3.18 are written to the database and, when hardware is connected, converted into a sorting command as described in Section 3.17.

[TO BE SUPPLIED: Figure 3.7 – flowchart of the real time detection and classification process]

Figure 3.7: Real time tomato detection and classification process

3.15 Hardware Prototype Development

The conveyor frame was constructed from 25 mm metal box-section bar to dimensions first validated in a three-dimensional Computer Aided Design (CAD) model, and painted for durability. The belt is driven by a NEMA 23 stepper motor coupled to the drive roller through a 20-tooth motor pulley, a 60-tooth roller pulley and a closed-loop timing belt, giving a 3:1 reduction, with six bracket-and-bearing assemblies supporting roller rotation. The motor is commanded through a DM542 driver over a step-and-direction interface at 0 to 2,000 steps per second, with belt speed set by a potentiometer read by the microcontroller. An adjustable single-axis camera mount allows camera position and angle to be tuned on the assembled rig.

Four infrared sensors and four servo-actuated gates are installed, one pair for each ripening class. The defect class is intentionally not assigned a gate: fruit classified as defective receives no actuation command and continues to the end of the belt into a single reject stream, which removes one gate from the mechanism without loss of function.

The physical arrangement along the belt is important to the control logic of Section 3.17 and is therefore stated explicitly. Working downstream from the camera, the gates are encountered in the order red, turning, breaker, green. Each gate has an infrared sensor mounted at its own position, immediately upstream of the diverter blade. Each servo is driven from a dedicated digital output and each sensor read on a dedicated digital input; the class-to-servo assignment is fixed in firmware and cross-checked against the mapping used by the host application. Each gate rests at 180° and rotates to 110° to divert, a sweep of 70°, returning automatically after a hold interval.

The controller is a single ESP32 board, powered from a 24 V supply on the driver side and a 12 V adapter on the logic side, so that stepper and servo transients cannot disturb the logic supply. The pin assignment and wiring diagram are given in Appendix C.

[TO BE SUPPLIED: Figure 3.8 – CAD model and photographs of the assembled conveyor prototype]

Figure 3.8: Computer aided design model and assembled hardware prototype

3.16 Software System Development

The software system is a Python application built around a web-based operator dashboard, supported by a detection module and a local relational database. The dashboard presents a live camera feed with drawn bounding boxes; a session panel showing tracker state, the running count of fruits processed and the class and confidence of the most recent fruit; an indicator panel with a selectable time window reporting total fruits processed, throughput, defect ratio and mean estimated shelf life; a class-distribution chart; and a table of recent detections.

The model is loaded once and cached for the session, with a selector for choosing among trained checkpoints, and each frame is run with an operator-adjustable confidence threshold defaulting to 0.45. Every finalised classification is written to the database with its confidence, bounding box, timestamp and shelf-life value, and the indicator panel reads from that same database, so that the displayed statistics and the stored record cannot diverge.

[TO BE SUPPLIED: Figure 3.9 – software architecture diagram and dashboard screenshot]

Figure 3.9: Software architecture and operator dashboard of the detection and sorting application

3.17 Hardware and Software Integration

3.17.1 Command Transfer

The dashboard communicates with the controller over a Bluetooth Serial Port Profile (SPP) link at 115,200 baud. On classification, the host transmits a plain-text command: one of four class commands corresponding to the four ripening gates, or a no-class command for fruit classified as defective.

Each command carries a monotonically increasing sequence number, and the host retransmits it until a matching acknowledgement is received, up to four attempts with a 0.5 second timeout each. The controller records the sequence number it last acted upon; a command whose sequence number matches that record is acknowledged again but not acted on a second time. Both halves of this protocol are necessary, and for asymmetric reasons. A lost command would silently remove a fruit from the queue, and a lost acknowledgement would cause the same fruit to be entered twice; either fault misaligns every gate assignment that follows it, so the error is persistent rather than transient.

3.17.2 Gate Release

On the controller side the firmware maintains a cascaded queue with one stage per gate position, ordered as the fruit encounters them along the belt. Every classification enters the first stage. When the infrared sensor at a stage registers the arrival of a fruit, the entry at the head of that stage's queue is removed and compared with the class served by that gate. If the two match, the gate is released and the fruit is diverted. If they do not, the entry is passed forward into the next stage's queue, to be tested again at the next gate. Belt order is preserved from end to end, so a first-in, first-out queue at each stage

is sufficient to keep entries aligned with the fruit they describe. Queue depth allows several fruits to be in transit simultaneously.

Releasing each gate from its own presence sensor, rather than from an elapsed time computed over the full camera-to-gate distance as in Equation (2.1), is the central design decision of the integration. It removes the dominant and distance-accumulating source of timing error, which is what makes the last gate in a timed system the least reliable. A short residual offset remains between the sensor and the diverter blade itself, and the firmware compensates for it with a release delay that is scaled according to the commanded belt speed, together with a fixed manual trim; the gate then holds open for 2,000 ms before returning to rest.

[TO BE SUPPLIED: Figure 3.10 – communication flow from classification through the staged queue to gate servo actuation]

Figure 3.10: Hardware and software integration and communication flow

3.18 Shelf Life Observation Procedure

Shelf-life estimation is derived from a time-series observation study rather than from a learned predictive model. Padma tomatoes of known initial ripening stage were held under room-temperature conditions and observed each morning over a continuous 31-day period. At each observation, colour differentiation was used to determine the fruit's position in the four-stage progression defined in Table 3.7, and the number of days spent in each stage was recorded. A fruit was considered to have left the study once it became unsellable, defined as the onset of excessive softening, fungal or rotting development, or severe surface blemishing.

[TO BE SUPPLIED: number of fruits observed in total and at each starting stage; the calendar dates of the observation period; the daily observation time; and the temperature and relative humidity of the storage environment. The published comparison study recorded 29 °C and 80 % relative humidity, so recording both permits a direct comparison in Chapter 4; without them the protocol is not reproducible]

From the recorded stage durations, the remaining shelf life associated with each detected class is computed by Equation (3.7) as the sum of the durations of that stage and all subsequent stages.

$$L(k) = \sum_{j=k}^{4} d_j \tag{3.7}$$

where k is the index of the detected stage, d_j is the measured duration of stage j, and stages are indexed 1 to 4 from green to red. Fruit classified as defective is assigned $L = 0$ by definition rather than by measurement, since it is rejected at the point of sorting.

Equation (3.7) uses the full duration of the detected stage, because the point at which a fruit entered that stage is not observable from a single image. The value it returns is therefore an upper bound on remaining shelf life, and is reported as such. The resulting per-class values are implemented in the system as a static lookup applied at the moment of classification, and the complete observation record is given in Appendix B.

3.19 Complete End to End System Process

The procedures described above combine into a single process, shown in Figure 3.11. Fruit is collected and pre-sorted, photographed under controlled illumination, annotated, preprocessed and augmented, and split into three subsets; the detector is then trained, validated and tested. In deployment, the live feed is passed frame by frame through the trained model, each fruit is tracked and classified by Equations (3.5) and (3.6), the finalised class is converted into a sorting command, the corresponding gate servo is released when the fruit reaches that gate's sensor, and the classification, confidence and estimated remaining shelf life are logged and displayed.

[TO BE SUPPLIED: Figure 3.11 – complete end to end system workflow from fruit input to sorted output and logged record]

Figure 3.11: Complete end to end system workflow from fruit input to sorted output and logged record

3.20 Experimental Procedure

Five experiments were designed to evaluate the system.

Experiment 1 — Held-out test evaluation. Quantifies detection and classification performance on unseen data, using the dataset of Section 3.6 and the procedure of Sections 3.12 and 3.13. The independent variable is the trained checkpoint; the dependent variables are precision, recall, mAP@50 and mAP@50–95, per class and overall; the controlled variables are the fixed test split and fixed evaluation settings. A single evaluation pass is performed by design, because repeated evaluation against the same split introduces selection bias.

Experiment 2 — Architecture comparison. Compares the deployed YOLOv8m model against a YOLO26m model trained on the identical dataset and evaluated on the identical held-out split, in order to justify the deployed architecture on evidence from this dataset. The independent variable is the architecture; the dependent variables are as in Experiment 1, reported with the bootstrap intervals of Section 3.13.

Two differences between the runs are not controlled and are reported alongside the result rather than left implicit: the comparison model was trained with a fixed batch size of two, where the deployed model used automatic batch selection, and the two runs early-stopped after different numbers of epochs. The comparison therefore establishes which of two concretely trained models performs better on this task, not which architecture is superior under matched optimisation budgets. Stating this is preferable to presenting the difference as a clean architectural result.

Experiment 3 — Live classification validation. Validates classification outside the offline test split, by presenting fruits of known class to the live camera inside the lightbox and recording the classification returned by the dashboard. This experiment was the one that exposed the confound described in Section 3.8.3, and it was repeated after correction. The dependent variable is the proportion of presented fruits classified correctly.

Experiment 4 — Conveyor transport and end-to-end sorting. Operates the conveyor under programmed stepper control with fruit of known class placed on the belt. It is

conducted in two parts: a mechanical part with no vision or sorting logic active, in which the dependent variable is the stability and consistency of transport; and a closed-loop part with the full pipeline active, in which the dependent variable is the proportion of fruits diverted at the correct gate.

Experiment 5 — Shelf-life observation. The study of Section 3.18. The dependent variable is the number of days each fruit spends in each ripening stage.

Raw records for all five experiments are given in Appendix B.

3.21 System Testing Procedure

System testing was planned in four phases: controlled-environment testing under light-box illumination; stress testing with varied fruit orientation, occlusion, capture distance and illumination; validation under realistic operating conditions; and comparison of automated grading against manual grading. The test cases derived from this plan are listed in Table 3.10 with the result expected of each; outcomes and pass or fail status are reported in Chapter 4.

Table 3.10: System test cases and expected results

ID	Test description	Input	Expected result
T-01	Held-out test set detection and classification accuracy	777 unseen labelled images	High precision, recall and mean average precision across all five classes
T-02	Architecture comparison on the same held-out split	Two trained checkpoints	A defensible basis for the deployed architecture choice
T-03	Live classification inside the lightbox	Known-class fruit, live camera	Correct class reported for each fruit presented
T-04	Conveyor belt mechanical transport	Fresh fruit on the running belt	Smooth and stable transport without stalling or displacement
T-05	Non-tomato object rejection	Frame containing fruit and distractor objects	Distractors not classified into any tomato class
T-06	Single gate actuation bench test	Direct command to one gate, belt stationary	Corresponding servo opens and returns automatically
T-07	End-to-end sorting accuracy	Known-class fruit moving on the running belt	Fruit physically diverted at the correct gate
T-08	Wireless command reliability under retry	Sequenced class commands over the live link	Commands delivered exactly once despite intermittent packet loss

These cases span detection-accuracy testing (T-01, T-02), classification testing (T-03), functional and mechanical testing (T-04, T-06) and integration testing (T-05, T-07, T-08).

3.22 Ethical and Safety Considerations

This research involved food produce and electromechanical prototype hardware. It did not involve human or animal subjects, the collection of personal data, or any procedure requiring institutional ethics board review, so the relevant considerations were engineering safety and food-handling practice.

- **Electrical safety.** The prototype operates a 24 V driver supply, a 12 V logic supply and the servo supply; wiring and connections were inspected before each

powered test.

- **Mechanical safety.** The belt, rollers, motor and gate servos present pinch-point hazards, and the belt area was kept clear of hands during all powered tests.

- **Food handling.** Fruit used for photography and conveyor testing was handled and disposed of hygienically. No chemical treatment or destructive testing was performed beyond the shelf-life observation study of Section 3.18, in which fruit degrades to spoilage by design.

- **Data and intellectual property.** Images originally obtained from a public dataset were used under that dataset's terms and were subsequently removed, as described in Section 3.8.3. The project's own annotated dataset is released under a Creative Commons Attribution 4.0 licence.

Chapter 4

RESULTS AND DISCUSSION

4.1 Detection Performance on the Held-Out Test Split

The selected checkpoint was evaluated once against the 777-image held-out test split, containing 787 annotated instances. Results are given in Table 4.1.

Table 4.1: Held out test set performance of the deployed YOLOv8m model

Class	Images	Instances	Precision	Recall	mAP@50	mAP@50–95
Breaker	191	191	0.924	0.937	0.966	0.762
Defect	45	66	0.766	0.818	0.804	0.599
Green	191	191	0.976	0.990	0.982	0.722
Red	190	190	0.987	0.958	0.988	0.724
Turning	149	149	0.880	0.906	0.941	0.749
Overall	**777**	**787**	**0.906**	**0.922**	**0.936**	**0.711**

[TO BE SUPPLIED: add the bootstrap 95 % confidence interval for each overall metric, computed as described in Section 3.13. This matters most for the defect row, which rests on 66 instances]

Two patterns in Table 4.1 deserve comment.

The three classes at the ends of the colour progression, green and red, are detected most reliably, at mAP@50 of 0.982 and 0.988. The classes in the middle of the progression perform less well, and the defect class least well of all. This ordering is not incidental. Green and red are defined by the absence and the near-completeness of red colouration respectively, which are absolute rather than relative judgements, whereas breaker and turning are separated from their neighbours by a threshold on a continuous quantity. A boundary placed at 30 % or 80 % red surface has no visual discontinuity behind it, so fruit close to either boundary is genuinely ambiguous rather than merely difficult.

The gap between mAP@50 and mAP@50–95 across all classes, from 0.936 to 0.711 overall, indicates that the model finds fruit reliably but localises the box less tightly. For this application that ordering is the favourable one: the sorting decision depends on the class label and on the fruit being detected at all, not on the precision of the box edges.

[TO BE SUPPLIED: Figure 4.1 – confusion matrix for the test set evaluation, raw counts and row-normalised. Source files: runs_local/tomato_5class_v6_balanced/test_eval/confusion_matrix.png and confusion_matrix_normalized.png]

Figure 4.1: Confusion matrix for the held out test set evaluation

[TO BE SUPPLIED: interpret Figure 4.1 in one paragraph. State explicitly which class pairs account for the largest off-diagonal mass, and confirm whether the dominant confusion is between adjacent ripening classes rather than with the defect class]

[TO BE SUPPLIED: Figure 4.2 – training and validation loss curves and detection metrics against epoch. Source file: runs_local/tomato_5class_v6_balanced/results.png]

Figure 4.2: Training and validation curves for the deployed model

[TO BE SUPPLIED: Figure 4.3 – precision, recall, F1 and precision-recall curves against confidence threshold. Source files: BoxP_curve.png, BoxR_curve.png, BoxF1_curve.png, BoxPR_curve.png]

Figure 4.3: Detection metric curves as a function of confidence threshold

Training converged in 4.57 hours. Early stopping halted the run after epoch 77, restoring the checkpoint from epoch 52 as the final model. The fused model comprises 93 layers and 25,842,655 parameters at 78.7 GFLOPs, and processes a test image in 6.7 ms of inference time, with 0.7 ms of preprocessing and 0.7 ms of postprocessing, on the RTX 3070 Ti host. At 30 FPS the camera delivers a frame every 33 ms, so inference is not the limiting factor in the real-time path.

Figure 4.3 is also the basis of the deployment confidence threshold of 0.45 used in Section 3.14, taken as the approximate maximum of the F_1 curve.

4.2 Architecture Comparison

Table 4.2 compares the deployed YOLOv8m model with a YOLO26m model trained on the identical dataset and evaluated on the identical held-out split.

Table 4.2: Comparison of YOLOv8m and YOLO26m on the same held out test split

Metric	**YOLOv8m**	**YOLO26m**	**Difference**
Precision	0.906	0.913	+0.007
Recall	0.922	0.911	−0.011
mAP@50	0.936	0.930	−0.006
mAP@50–95	0.711	0.785	+0.074

Table 4.3: Per class comparison of YOLOv8m and YOLO26m on the same held out test split

	mAP@50			**mAP@50–95**		
Class	**v8m**	**26m**	**Diff.**	**v8m**	**26m**	**Diff.**
breaker	0.966	0.962	−0.004	0.762	0.845	+0.083
defect	0.804	0.754	−0.050	0.599	0.681	+0.082
green	0.982	0.991	+0.009	0.722	0.816	+0.094
red	0.988	0.985	−0.003	0.724	0.768	+0.044
turning	0.941	0.956	+0.015	0.749	0.785	+0.036

The two architectures are effectively tied on detection rate: the mAP@50 difference of -0.006 overall is within the range that would be expected from run-to-run variation on a test split of this size. Localisation quality differs clearly and consistently, with YOLO26m ahead on mAP@50–95 in every class and by 0.074 overall.

The exception is instructive. The defect class is the only class where YOLO26m loses ground on detection rate, falling from 0.804 to 0.754, while simultaneously gaining 0.082 on mAP@50–95. The class that improves least in detection is the same class that was reconstructed after the confound described in Section 3.8.3, has the fewest real training instances, and relies most heavily on the augmentation supplement. That is consistent with the defect class remaining the most fragile part of the dataset rather than with any deficiency of the architecture.

YOLOv8m was retained for deployment. The metric on which YOLO26m leads, mAP@50–95, rewards tight box regression, which the sorting mechanism does not use: the gate decision depends on the class label and on the fruit being detected, not on box edge precision. On the metrics that do bear on sorting, detection rate and defect-class recall, YOLOv8m is equal or ahead. This is a task-specific justification rather than a claim that one architecture is generally superior.

[TO BE SUPPLIED: add the bootstrap confidence intervals from Section 3.13 to Table 4.2, then state which of these differences survive. Several of the mAP@50 differences are likely to fall inside overlapping intervals, and saying so strengthens the argument rather than weakening it]

The uncontrolled differences between the two training runs, noted in Section 3.20, must be carried into the interpretation. The comparison model was trained with a fixed batch size of two against automatic batch selection for the deployed model, and the runs early-stopped after different numbers of epochs. The result therefore establishes which of two concretely trained models better suits this task, and not which architecture is superior under matched optimisation budgets.

4.3 Effect of the Dataset Corrections

[TO BE SUPPLIED: report the defect-class performance before and after the two corrections of Section 3.8.3, so that the corrective cycle is evidenced rather than asserted. The pre-correction run on the earlier dataset version is retained at runs_local/tomato_5class_local/. Give defect mAP@50 before the box-geometry fix, after it, and after the imaging-conditions fix, and identify which dataset version each figure belongs to]

This section is the most transferable result in the chapter and should be written at length. A detector that scored well on an offline split was failing on live input for a reason no aggregate metric exposed, and the diagnosis came from the confusion matrix and live trials rather than from the summary table. The general lesson, that a scarce class topped up from an outside source can teach a network the capture conditions instead of the object, applies well beyond tomato grading.

4.4 Live Classification Validation

[TO BE SUPPLIED: report Experiment 3. Give the number of fruits presented per class, the number classified correctly, and the resulting per-class and overall accuracy, for both the pre-correction and post-correction sessions. Screenshot evidence is held in live dashboard results-2026.07.26 and live dashboard results-2026.07.29]

4.5 Conveyor Transport and End-to-End Sorting

[TO BE SUPPLIED: report Experiment 4. Part one: the mechanical transport trial, with the number of fruits run, the belt speed used, and any stalling or displacement observed. Part two: the closed-loop sorting trial, with the number of fruits per class, how many were diverted at the correct gate, and the failure mode of each incorrect diversion. Report the measured belt speed in cm/s, since it is needed to interpret the gate release delay of Section 3.17]

4.6 Shelf Life Observation

[TO BE SUPPLIED: report Experiment 5. Give the measured duration of each ripening stage with its spread across fruits, not a single value per stage, and the resulting remaining shelf life per class from Equation (3.7)**. Then compare against the published Padma values reproduced in Table 2.1, noting that the six published stages map onto your four by the correspondence in Table 3.6. Agreement or disagreement with that study is a result in itself and should be stated either way]**

4.7 System Test Outcomes

[TO BE SUPPLIED: reproduce Table 3.10 with two further columns, actual result and pass or fail status, one row per test case T-01 to T-08. Do not enter a pass status for any row not backed by a recorded trial]

4.8 Discussion

[TO BE SUPPLIED: draw the results together. Suggested structure: (i) what the detection results show about the feasibility of cultivar-specific grading for the Padma variety, and how they compare with the published tomato detection results reviewed in Section 2.1.2; (ii) why the ripening-boundary confusion is a property of the class definition rather than a model deficiency, and what that implies for anyone defining discrete classes on a continuous colour gradient; (iii) what the sensor-released gate design achieved in practice against the timed alternative of Equation (2.1)**; (iv) how the measured shelf-life values compare with the published Padma study; and (v) what the corrective cycle of Section 4.3 implies for dataset construction practice more generally]**

Chapter 5

CONCLUSIONS

5.1 Summary of Findings

[TO BE SUPPLIED: one paragraph per research question from Section 1.2.1, each answered directly from the results of Chapter 4 and each citing the table or figure that supports it. Answer the question that was asked; do not restate the method]

5.2 Achievement of Objectives

[TO BE SUPPLIED: take the six specific objectives of Section 1.3 in order and state for each whether it was fully achieved, partially achieved or not achieved, with the evidence. Objectives that were only partially met should be stated as such – an examiner reads an honest partial far more favourably than an unsupported claim of completion]

5.3 Limitations

The following limitations follow from evidence gathered during the study rather than from generic caution.

1. **Ripening classes are cut from a continuous variable.** Breaker, turning and green are separated by thresholds on the proportion of red surface, and fruit near a threshold is genuinely ambiguous. The residual confusion between adjacent ripening classes is therefore a property of a hard multi-class framing applied to a continuous quantity, not a deficiency the model could be trained out of.

2. **Class boundaries were judged visually, without instrumental measurement.** No colorimetric measurement was used to set or verify the 30 % and 80 % bound-

aries at annotation time. **[TO BE SUPPLIED: state the annotation reliability figure from Section 3.8.3 here, or state that reliability was not measured]**

3. **Splitting was performed by image, not by fruit. [TO BE SUPPLIED: resolve against Section 3.9. If images of one physical fruit could fall in more than one split, state plainly that the reported metrics may be optimistically biased and quantify the exposure if possible]**

4. **Single-file flow is assumed.** The tracking and voting policy of Section 3.14 assumes one fruit is in view at a time. Simultaneous multi-fruit identity tracking is not implemented, which caps achievable throughput relative to a system that maintains per-object identity.

5. **One capture setting.** All training images were captured under a single lightbox arrangement at one collecting centre. Performance under substantially different lighting or background beyond what Experiment 3 validated has not been quantified.

6. **Shelf life is a static per-class lookup, not a per-fruit prediction.** The value returned is a fixed figure derived from measured stage durations, applied by Equation (3.7) and conditioned only on the detected class. It is not an estimate conditioned on the condition of the individual fruit, and because the point of entry into the detected stage is unobservable it is an upper bound rather than an expected value.

7. **Defect class evidence is thin.** The defect class contributes only 66 of the 787 test instances and depends on an augmentation supplement for training balance, so its reported figures carry the widest uncertainty of any class.

8. **[TO BE SUPPLIED: add any limitation arising from the end-to-end sorting trial reported in Section 4.5, once that trial is written up]**

5.4 Recommendations for Future Work

[TO BE SUPPLIED: develop each of the following into a short paragraph, keeping each one traceable to a specific limitation above rather than offering generic

suggestions]

1. **Treat ripeness as ordinal rather than nominal.** An ordinal regression head, or a model predicting the proportion of red surface directly with thresholds applied afterwards, would match the underlying continuous variable and would make a boundary error between adjacent stages cost less than a two-stage error, which a flat five-way classifier cannot express.

2. **Ground the class boundaries instrumentally.** Recording a colorimetric measurement for a subset of fruit at annotation time would convert the 30 % and 80 % boundaries from visual judgements into measured ones.

3. **Split by fruit identity.** Assigning every image of one physical fruit to a single split removes the near-duplicate leakage pathway entirely.

4. **Extend the capture domain.** Deliberately capturing under varied illumination and background, and re-evaluating, would quantify the robustness that the present single-setting dataset cannot.

5. **Condition shelf life on the individual fruit.** Predicting remaining shelf life from fruit-level visual features rather than from class membership alone would replace the upper bound of Equation (3.7) with a per-fruit estimate.

6. **Multi-object tracking.** Maintaining per-fruit identity across frames would lift the single-file constraint and raise achievable throughput.

5.5 Concluding Remarks

[TO BE SUPPLIED: close in one short paragraph: what was built, what it demonstrates for post-harvest handling of the Padma variety in Sri Lanka, and what the study contributes beyond this cultivar. The corrective cycle documented in Section 4.3 is the most transferable contribution and is worth naming here]

References

Abekoon, T., Sajindra, H., Jayakody, J. A. D. C. A., Samarakoon, E. R. J. and Rathnayake, U. (2024), 'Image processing techniques to identify tomato quality under market conditions', *Smart Agricultural Technology* **7**, 100433.

Allo, Y. M. K., Paendong, I. P. and Saputro, P. H. (2025), 'Classification of tomato ripeness levels using convolutional neural network (cnn)', *Journal of Intelligent Systems and Information Technology* **2**(2), 80–87.

Borba, K. R., Aykas, D. P., Milani, M. I., Colnago, L. A., Ferreira, M. D. and Rodriguez-Saona, L. E. (2021), 'Portable near infrared spectroscopy as a tool for fresh tomato quality control analysis in the field', *Applied Sciences* **11**(7), 3209.

Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M. and Kalinin, A. A. (2020), 'Albumentations: Fast and flexible image augmentations', *Information* **11**(2), 125.

Centino, M. C., Pitogo, V. A. and Pacot, M. P. B. (2025), 'Tomato maturity assessment: Using convolutional neural networks and image processing-based domain knowledge', *AEIS* **1**(1), 34–45.

Ding, J., Zou, Y., Wang, Y., Han, L., Xiao, Y., Zhang, R. and Xi, X. (2026), 'A lightweight task-adaptive YOLO for tomato ripeness detection in complex orchard environments', *Horticulturae* **12**, 805.

Dwyer, B., Nelson, J. and Hansen, T. (2024), 'Roboflow (version 1.0) [computer software]', `https://roboflow.com`.

Etikan, I., Musa, S. A. and Alkassim, R. S. (2016), 'Comparison of convenience sampling and purposive sampling', *American Journal of Theoretical and Applied Statistics* **5**(1), 1–4.

Fan, X. and Chai, X. (2026), 'TRD-Net: An efficient tomato ripeness detection network based on improved yolo v8 for selective harvesting', *Frontiers in Plant Science* **17**, 1748741.

Garcia, M. B., Ambat, S. and Adao, R. T. (2019), Tomayto, tomahto: A machine learning approach for tomato ripening stage identification using pixel-based color image classification, *in* 'Proceedings of the IEEE 11th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM)', IEEE, pp. 1–6.

Geetha, G., Prabhu Kumar, P. C., Suriyaraj, V. and Naveen Kumar, J. (2025), 'Smart prediction of shelf life and tomato sorting using deep learning', *Asian Journal of Advances in Agricultural Research* **25**(10), 53–67.

Huang, W., Liao, Y., Wang, P., Chen, Z., Yang, Z., Xu, L. and Mu, J. (2025), 'AITP-YOLO: Improved tomato ripeness detection model based on multiple strategies', *Frontiers in Plant Science* **16**, 1596739.

Ifmalinda, Andasuryani and Rasinta, I. (2023), 'Identification of tomato ripeness levels (*Lycopersicum esculentum* mill.) using android-based digital image processing', *IOP Conference Series: Earth and Environmental Science* **1182**, 012003.

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. and Zitnick, C. L. (2014), Microsoft COCO: Common objects in context, *in* 'Computer Vision – ECCV 2014', Springer, pp. 740–755.

Liu, G., Mao, S. and Kim, J. H. (2019), 'A mature-tomato detection algorithm using machine learning and color analysis', *Sensors* **19**(9), 2023.

Liu, Y., Wei, C., Yoon, S.-C., Ni, X., Wang, W., Liu, Y., Wang, D., Wang, X. and Guo, X. (2024), 'Development of multimodal fusion technology for tomato maturity assessment', *Sensors* **24**(8), 2467.

Moya, V., Guerra, M., Pazmiño, K., Abedrabbo, F., Chicaiza, F. A. and Pozo-Espín, D. (2025), 'Tomato classification with YOLOv8: Enhancing automated sorting and quality assessment', *Smart Agricultural Technology* **12**, 101221.

Nalupano, J. M. G., Omagap, M. P., Fortaleza, K. J. G., Ecraela, F. R. B. and Orquia, J. J. D. (2024), 'Classification of kinalabasa tomato using convolutional neural network', *Journal of Innovative Technology Convergence* **6**(3), 99–110.

Padilla, R., Netto, S. L. and da Silva, E. A. B. (2020), A survey on performance metrics for object-detection algorithms, *in* 'Proceedings of the 27th International Conference on Systems, Signals and Image Processing (IWSSIP)', IEEE, pp. 237–242.

Patria, L., Makhtar, M., Sambas, A., Multajam, R., Ayob, A. F. M., Jamaludin, S., Sanjaya, W. S. M. and Chuan, O. Y. (2025), 'Development of a vision-based mobile robot with artificial neural networks (ann) for classification of tomato ripeness', *Engineering Letters* **33**(6), 32–43.

Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016), You only look once: Unified, real-time object detection, *in* 'Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)', IEEE, pp. 779–788.

Safira, I., Fuadi, W. and Rosnita, L. (2026), 'Performance of YOLOv8 algorithm and real-time detection transformer in tomato ripeness detection system', *Journal of Artificial Intelligence and Engineering Applications* **5**(3).

Santoso, K. A., Kamsyakawuni, A. and Izza, S. V. R. (2024), 'Comparison classification of tomatoes ripeness based on rgb, hsv and cmyk colors based on correlation coefficient', *Journal of Computers and Digital Business* **3**(3), 112–120.

Saxena, A., Agarwal, A., Nagrath, B., Jayavanth, C. S., Thulasidoss, S., Maheswari, S. and Sasikumar, P. (2025), 'Deep learning-driven IoT solution for smart tomato farming', *Scientific Reports* **15**, 31092.

Shorten, C. and Khoshgoftaar, T. M. (2019), 'A survey on image data augmentation for deep learning', *Journal of Big Data* **6**, 60.

Tarlak, F. (2023), 'The use of predictive microbiology for the prediction of the shelf life of food products', *Foods* **12**(24), 4461.

Terven, J. and Cordova-Esparza, D. (2023), 'A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS', *Machine Learning and Knowledge Extraction* **5**(4), 1680–1716.

United States Department of Agriculture (1991), 'United states standards for grades of fresh tomatoes'.

Wang, D., Fang, Z., Mo, M., Gan, J. and Sun, Z. (2026), 'Tomato ripeness detection method based on improved YOLOv11 lightweight model', *Frontiers of Agricultural Science and Engineering* **13**(3), 25657.

Waseem, M., Sajjad, M. M., Naqvi, L. H., Majeed, Y., Rehman, T. U. and Nadeem, T. (2025), 'Deep learning model for precise and rapid prediction of tomato maturity based on image recognition', *Food Physics* **2**, 100060.

Yang, Z., Li, Y., Han, Q., Wang, H., Li, C. and Wu, Z. (2025), 'A method for tomato ripeness recognition and detection based on an improved yolov8 model', *Horticulturae* **11**(1), 15.

APPENDICES

All supporting materials which are too extensive for the body text are included here.

Appendix A: Sample Records and Detailed Annotation Procedure

[TO BE SUPPLIED]

- Sample-level record: total number of fruits photographed, the number per class, and the capture session dates (referred to in Section 3.3.1).
- Inclusion and exclusion criteria applied to each class at the point of collection.
- The full Roboflow annotation procedure: tool navigation, box-drawing convention, class-labelling rules and the review step (referred to in Section 3.8).
- The annotation reliability check: sample size, procedure and agreement statistic (referred to in Section 3.8.3).

Appendix B: Training Configuration, Raw Experimental Records and Shelf Life Data

[TO BE SUPPLIED]

- The complete training configuration file, `args.yaml`, for the deployed run (referred to in Section 3.12).
- The full per-epoch metric log, `results.csv`, and the training console log.
- The equivalent configuration and log for the comparison architecture of Experiment 2.

- Raw records for Experiments 3 and 4: trial dates, fruit counts per class, per-trial outcomes and the dashboard screenshots.
- The complete shelf-life observation record from Experiment 5: per-fruit, per-day stage assignment, and the storage temperature and relative humidity (referred to in Section 3.18).

Appendix C: Pin Assignment, Wiring Diagrams and Component Data

[TO BE SUPPLIED]

- The ESP32 pin assignment table: the digital output driving each gate servo, the digital input reading each infrared sensor, the step and direction pins to the motor driver, and the analogue input for the speed potentiometer (referred to in Sections 3.3.2 and 3.15).
- The physical layout diagram showing gate and sensor positions along the belt in the order red, turning, breaker, green, together with the sensor-to-blade offset measured at each gate.
- Wiring diagrams for the 24 V driver domain, the 12 V logic domain and the servo supply.
- Datasheets or specifications for the minor electronic components omitted from Table 3.2.
- The firmware source listing for the staged queue and command protocol described in Section 3.17.